Centralized Research Workspace

An Architectural Deep Dive into a Layered, Secure, Multi-Tenant Research Platform

Team noWayHome | Rafat Abdullah · Mahiul Kabir · Sohom Sattyam

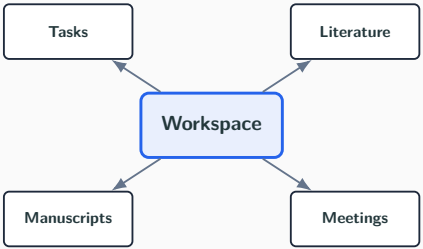
CSE 4636 — Web Architecture

What Is CRW?

The problem: academic researchers juggle separate tools for literature review, task tracking, manuscript drafting, and meeting notes — constant app-switching fatigue.

The solution: one collaborative workspace, four integrated modules, one consistent access-control model.

- Kanban / list task tracking
- Literature repository with threaded annotations
- Versioned manuscript drafting (Markdown)
- Meeting minutes & decisions log
- Cross-module global search



The Workspace is the tenancy boundary every module hangs off of.

Why This Deck Is About Architecture

This talk deliberately skips feature tours and focuses on how the system is built:

1. A strict **layered backend** (Controller → Service → Repository → Entity)
2. An **API-contract layer of DTOs** that never leaks persistence models
3. **Stateless JWT security** composed with **method-level authorization**
4. A hand-rolled **multi-tenancy guard** enforcing workspace-scoped access
5. Centralized **exception handling** for a uniform error contract
6. A **React context + interceptor** architecture on the frontend
7. A single **cross-cutting search** that respects all of the above

Technology Stack

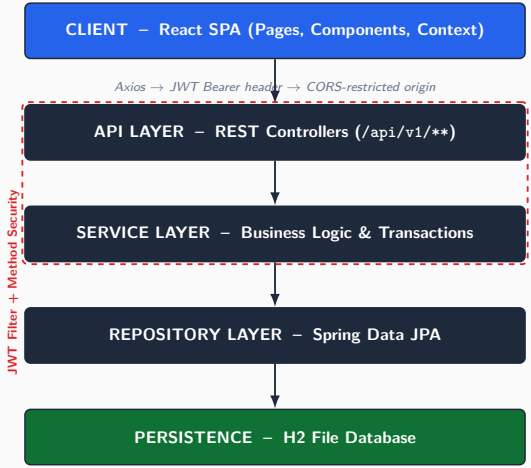
Backend

- Java 17, Spring Boot 3.3
- Spring Data JPA (Hibernate)
- Spring Security 6 (JWT, `@EnableMethodSecurity`)
- H2 — **file-backed**, not in-memory
- JJWT 0.12, Lombok, Bean Validation

Frontend

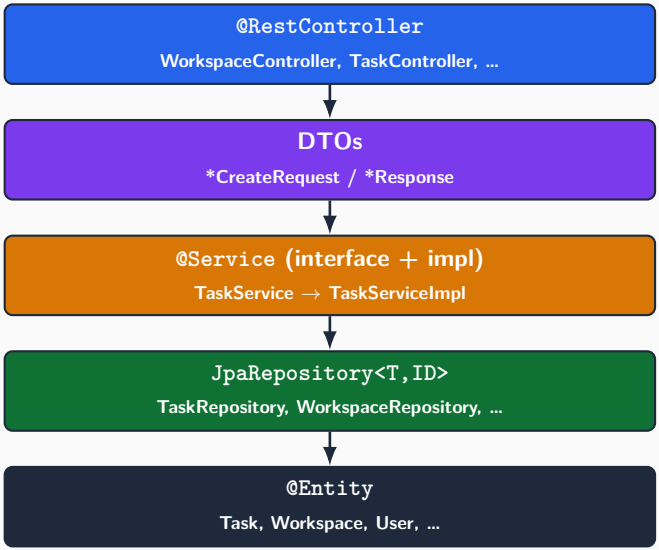
- React 18 + Vite
- React Router (protected routes)
- Axios with request interceptors
- React Context for global state
- React Markdown for rendering

System Architecture — 30,000 ft View



Every horizontal boundary is a contract: DTOs cross the API boundary, entities never do.

Backend: Strict Layered Design



Why it matters:

- Every domain (*Task*, *Literature*, *Manuscript*, *Meeting*, *Workspace*, *User*, *Chat*) repeats the same 5-layer skeleton
- Controllers stay thin: validate input, delegate, map to DTO
- Services own transactions & business rules (e.g. reassigning a task, publishing a manuscript version)
- Interfaces (`TaskService`) decouple contracts from implementation — testable, swappable

Why DTOs? Isolating the API Contract

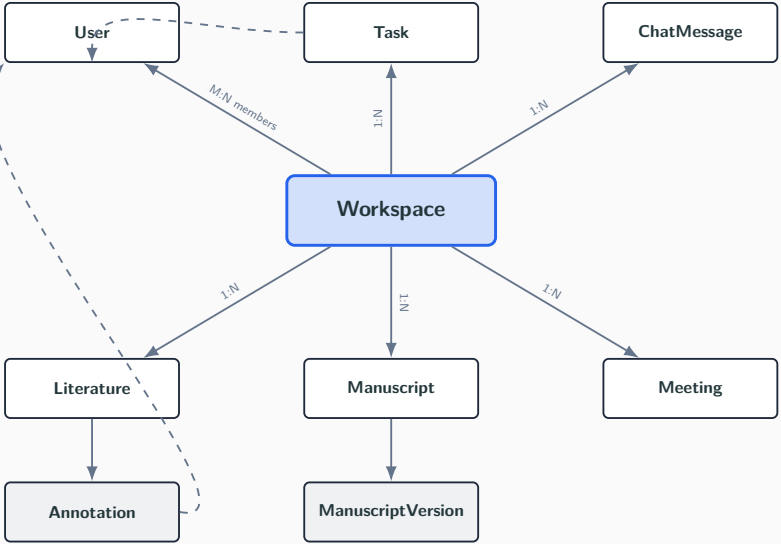
Without DTOs ×

- Entities serialized directly to JSON
- Lazy-loaded collections trigger `LazyInitializationException` or N+1 explosions
- Password hashes, internal IDs, JPA proxies leak to clients
- Any schema change breaks the frontend

With DTOs ✓

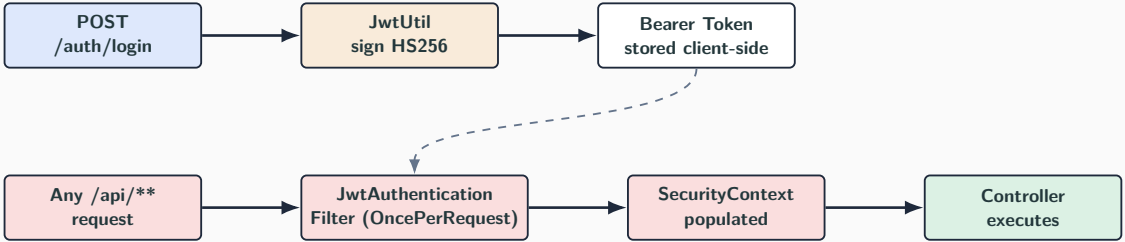
- `*CreateRequest` validates inbound payloads (`@NotBlank`, `@Valid`)
- `*Response` shapes exactly what the UI needs
- Entity graph can evolve independently of the wire format
- Sensitive fields (e.g. `User.password`) are structurally unreachable

Domain Model: Workspace as the Tenancy Root



Solid = ownership (`@OneToMany`/`@ManyToMany`) Dashed = attribution (`assignee`/`author`)

Security Architecture: Stateless JWT Flow



- `SessionCreationPolicy.STATELESS` — no server-side session, horizontally scalable by design
- Filter runs **before** `UsernamePasswordAuthenticationFilter`; validates signature + expiry once per request
- Passwords hashed with `BCryptPasswordEncoder`; CSRF disabled only for stateless `/api/**` & `/h2-console/**`

Authorization: Two Independent Guards

1. Role-based (coarse)

`@EnableMethodSecurity` + `@PreAuthorize("hasRole('ADMIN')")`

- Enforced declaratively at the method boundary
- Distinguishes `ADMIN` vs `RESEARCHER`
- e.g. only an admin can remove a member

2. Tenancy-based (fine)

`WorkspaceAccessGuard`

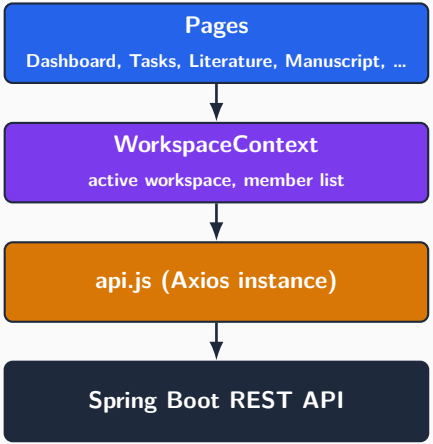
- Resolves the caller from `SecurityContext` → `User`
- `requireMember(workspace)` throws `AccessDeniedException` if the caller isn't in `workspace.members`
- Every service call that touches a Task/Literature/Manuscript/Meeting runs through this check first

Role security answers "*what can this user do*"; the workspace guard answers "*what is this user's data*".

Centralized Exception Handling

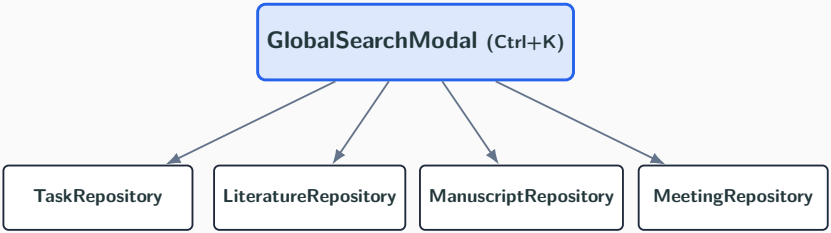
- Every controller is exception-free of boilerplate try/catch — one advice class maps exceptions to HTTP status codes
- `RestAuthenticationEntryPoint` guarantees unauthenticated calls get a clean 401 `JSON` body, not a servlet-container HTML page

Frontend Architecture: Context + Interceptor



- **Axios request interceptor** auto-attaches the JWT from `localStorage` to every call — no manual header wiring in components
- **WorkspaceContext** centralizes "which workspace am I in" so every page reads the same source of truth
- **ProtectedRoute** wraps authenticated pages; redirects unauthenticated users before a request is even attempted
- Mirrors the backend's separation: presentation (*Pages*) never talks to transport (*Axios*) directly

Cross-Cutting Concern: Global Search



- A single query fans out across four repositories, scoped to the **active workspace** — reuses the same `WorkspaceAccessGuard` check
- Demonstrates that the layered design **composes**: a feature spanning every module still respects one security boundary and one DTO contract
- Results are merged and ranked client-side before rendering in the modal

Persistence Strategy

File-backed H2

- `jdbc:h2:file:./data/crwdb`
- Data survives backend restarts — unlike a typical in-memory H2 demo setup
- Zero external DB dependency: clone, run, done
- `/h2-console` exposed (dev-only) for direct inspection

Cascading & ownership

- `CascadeType.ALL` + `orphanRemoval=true` on Workspace→Task, Manuscript→Version, Literature→Annotation
- Deleting a parent cleanly deletes its dependents — no orphaned rows
- Lazy fetch by default everywhere; DTO mapping decides what actually gets pulled

Architectural Principles, Distilled

- **Separation of concerns** — 5 uniform layers across 6 domains
- **Stateless security** — JWT, no server session, scales horizontally
- **Defense in depth** — role checks *and* tenancy checks, independently
- **Contract isolation** — DTOs shield entities from the wire format
- **Uniform failure handling** — one advice class, one error shape
- **Composable modules** — global search proves the layers reuse cleanly

A small feature set, built on patterns that scale to a much larger one.

Thank You

Questions?

`github.com/Rafat-Pantho/Centralized-Research-Workspace`

Team noWayHome — Rafat Abdullah (220041102) · Mahiul Kabir (220041109) · Sohom Sattyam (220041141)